Héberger un projet Symfony dans Coolify

Démarche en 3 parties, à suivre dans l'ordre :

- **A)** Adapter le projet Symfony
- **B)** Pousser les modifications sur GitHub
- **C)** Créer et configurer la ressource dans Coolify

A) Adapter le projet Symfony

1. `compose.yaml`

Remplacer le contenu de `compose.yaml` (à la racine du projet) par :

```yaml
services:
  php:
    build: .
    restart: unless-stopped
    environment:
      APP_ENV: prod
      APP_SECRET: ${APP_SECRET}
      TRUSTED_PROXIES: ${TRUSTED_PROXIES:-REMOTE_ADDR}
      DATABASE_URL: mysql://${MARIADB_USER:-app}:${MARIADB_PASSWORD}@database:3306/${MARIADB_D
      # Caddy écoute en HTTP sur le port 80 : le HTTPS est géré par Traefik (Coolify)
      SERVER_NAME: :80
    depends_on:
      database:
        condition: service_healthy

  database:
    image: mariadb:${MARIADB_VERSION:-11.4}
    restart: unless-stopped
    environment:
      MARIADB_DATABASE: ${MARIADB_DATABASE:-app}
      MARIADB_USER: ${MARIADB_USER:-app}
      MARIADB_PASSWORD: ${MARIADB_PASSWORD}
      MARIADB_ROOT_PASSWORD: ${MARIADB_ROOT_PASSWORD}
    healthcheck:
      test: ["CMD", "healthcheck.sh", "--connect", "--innodb_initialized"]
      timeout: 5s
      retries: 5
      start_period: 60s
    volumes:
      - database_data:/var/lib/mysql:rw

volumes:
  database_data:
```

Explications :

- `build: .` : l'image du service php est construite à partir du `Dockerfile` situé à la racine du projet (voir l'étape suivante).
- Aucun `ports:` n'est publié : Coolify (via son proxy Traefik) route le trafic vers le port 80 du service php.
- Les `${VARIABLE}` seront renseignées dans Coolify (partie C). `${VARIABLE:-valeur}` définit une valeur par défaut.

2. `Dockerfile`

Créer un fichier `Dockerfile` à la racine du projet, avec ce contenu :

```
FROM dunglas/frankenphp:1-php8.4

RUN install-php-extensions intl opcache pdo_mysql zip

COPY --from=composer:2 /usr/bin/composer /usr/bin/composer

ENV APP_ENV=prod \
    FRANKENPHP_CONFIG="worker ./public/index.php" \
    COMPOSER_ALLOW_SUPERUSER=1

WORKDIR /app
COPY . .

RUN composer install --no-dev --optimize-autoloader --no-scripts --no-interaction \
    && composer dump-env prod \
    && php bin/console importmap:install \
    && php bin/console asset-map:compile \
    && php bin/console cache:warmup \
    && mkdir -p var && chown -R www-data:www-data var

# Migrations puis démarrage du serveur
CMD ["sh", "-c", "php bin/console doctrine:migrations:migrate --no-interaction --allow-no-migr
```

Explications :

- Image de base FrankenPHP avec PHP 8.4 et installation des extensions PHP nécessaires (`pdo_mysql` pour MariaDB).
- `FRANKENPHP_CONFIG="worker ./public/index.php"` active le mode worker de FrankenPHP sur le point d'entrée de Symfony. `symfony/runtime` le détecte automatiquement.
- Le build installe les dépendances sans les paquets de dev, prépare la configuration de production, télécharge les bibliothèques JavaScript (`importmap:install`), compile les assets et réchauffe le cache.
- `importmap:install` est indispensable : `composer install --no-scripts` ne le lance pas, et le dossier `assets/vendor/` est dans le `.gitignore`, donc absent de GitHub. Sans cette commande, `asset-map:compile` échoue avec `The "@hotwired/stimulus" vendor asset is missing`.
- Au démarrage du conteneur, les migrations Doctrine sont exécutées, puis le serveur FrankenPHP est lancé.
- `--allow-no-migration` est nécessaire tant que le projet n'a aucune migration (dossier `src/Migrations/` vide) : sans lui, `doctrine:migrations:migrate` échoue avec `The version "latest" couldn't be reached, there are no registered migrations`, et à cause du `&&` le serveur ne démarre jamais.

3. `.dockerignore`

Créer un fichier nommé exactement `.dockerignore` à la racine du projet (à côté du `Dockerfile`), avec ce contenu :

```
.git/
.idea/
.vscode/
vendor/
var/
node_modules/
.env.local
.env.*.local
docs/
```

Explications :

- Ces fichiers ne sont pas copiés dans l'image par le `COPY . .` du `Dockerfile` : dépendances locales (réinstallées par `composer install`), cache, historique Git, et surtout secrets locaux.
- Ne pas ajouter `.env` : il est nécessaire au build (`composer dump-env prod`).

4. Trusted proxies

Dans `config/packages/framework.yaml`, ajouter sous `framework:` :

```
framework:
    trusted_proxies: '%env(TRUSTED_PROXIES)%'
    trusted_headers: ['x-forwarded-for', 'x-forwarded-host', 'x-forwarded-proto', 'x-forwarde
```

Puis ajouter dans le fichier `.env` (valeur par défaut, committée) :

```
TRUSTED_PROXIES=
```

Explications :

- Traefik (le proxy de Coolify) se place devant l'application et transmet les informations de la requête d'origine dans les en-têtes `X-Forwarded-*`. Sans cette configuration, Symfony les ignore et génère des URL en `http` au lieu de `https`.
- `%env(TRUSTED_PROXIES)%` exige que la variable existe, sinon Symfony s'arrête avec `Environment variable not found: "TRUSTED_PROXIES"`. D'où la ligne vide dans `.env` : en local, il n'y a pas de proxy.
- En production, `TRUSTED_PROXIES` vaut `REMOTE_ADDR` : le `compose.yaml` (étape 1) lui donne cette valeur par défaut, qui l'emporte sur `.env`. `REMOTE_ADDR` est un mot-clé de Symfony qui fait confiance à l'adresse IP qui se connecte directement au conteneur, c'est-à-dire Traefik. Cela convient car le conteneur n'est joignable que via le proxy de Coolify (aucun `ports:` publié).

B) Pousser les modifications sur GitHub

```
git status
git add compose.yaml Dockerfile .dockerignore config/packages/framework.yaml .env
git commit -m "Préparer le déploiement Coolify (FrankenPHP, MariaDB)"
git push origin main
```

Explications :

- Coolify déploie ce qui est sur GitHub : tous les fichiers créés ou modifiés en A doivent être poussés sur la branche `main`.
- `git status` permet de vérifier la liste des fichiers modifiés avant le `git add`.
- Le dépôt doit être **public**, car la ressource Coolify est de type *Public Repository*.
- Ne jamais committer de secrets (`.env.local`, mots de passe) : ils se définissent dans Coolify (partie C).

C) Procédure dans Coolify

1. Créer la ressource

1. **Projet** → choisir l'environnement (ex. *2026-sio2-portfolio*) → **+ New Resource**.
2. Choisir **Public Repository**.
3. Saisir l'URL du dépôt dans l'unique champ **Repository URL (https://)** : `https://github.com/LaurentBouquet/portfolio-coolify`, puis valider (**Check repository**).
4. Sur l'écran suivant, renseigner la configuration de la ressource :
   - **Branch** : `main`
   - **Build Pack** : **Docker Compose**
   - **Base Directory** : `/`
   - **Docker Compose Location** : `/compose.yaml`
5. Cliquer sur **Continue** : la ressource est créée et ouverte sur sa page **Configuration**.

2. Variables d'environnement

Dans la ressource, onglet **Environment Variables** :

1. Coolify détecte les `${VARIABLE}` du `compose.yaml` et les liste. Si une variable manque, cliquer sur **+ Add**.
2. Renseigner la valeur de chaque variable (**Key** / **Value**) :

Variable	Valeur
`APP_SECRET`	valeur aléatoire (clé secrète de Symfony)
`MARIADB_PASSWORD`	valeur aléatoire (mot de passe de l'utilisateur applicatif)
`MARIADB_ROOT_PASSWORD`	valeur aléatoire (mot de passe root MariaDB)

Ces trois valeurs sont des secrets : chacune doit être **générée de façon aléatoire et différente des deux autres**. Ne jamais copier/coller celles d'un exemple, d'un tutoriel ou d'un autre projet, ni les committer dans Git.

Pour générer une valeur aléatoire (valable sous Windows, Linux et macOS, car PHP est déjà installé pour Symfony) :

```
php -r "echo bin2hex(random_bytes(32));"
```

Lancer la commande **une fois par variable** et copier chaque résultat dans la valeur correspondante. `openssl rand -hex 32` fait la même chose, mais `openssl` n'est pas disponible par défaut sous Windows.

Aucun besoin de retenir ces valeurs : seule l'application les utilise, et elles restent stockées dans Coolify. Le format hexadécimal évite les caractères spéciaux qui casseraient l'URL `DATABASE_URL`.

> MariaDB ne lit `MARIADB_PASSWORD` et `MARIADB_ROOT_PASSWORD` qu'à la **première création** de la base (volume vide). Les modifier ensuite dans Coolify ne change pas les mots de passe déjà enregistrés dans la base.

`TRUSTED_PROXIES` (`REMOTE_ADDR`), `MARIADB_USER`, `MARIADB_DATABASE` et `MARIADB_VERSION` ont une valeur par défaut dans le `compose.yaml` : ne les définir que pour la changer.

> ATTENTION : Saisir la valeur aléatoire à droite du nom de la variable (pas dans le champ `Comment`) dans le champ avec un oeil.

3. Pour enregistrer chaque variable, cliquer sur le bouton **Update** situé sur sa ligne. Une variable saisie mais non enregistrée avec **Update** est perdue.

> Les variables ne sont prises en compte qu'au **prochain déploiement**.

3. Définir le sous-domaine

1. Dans l'onglet **General**, le champ **Domains** apparaît pour chaque service du compose. Pour le service `php`, saisir par exemple :

   ```
   https://portfolio-demo.app.ndlpavranches.fr
   ```

   Si le conteneur n'écoute pas sur le port 80, ajouter le port à la fin : `https://portfolio.app.ndlpavranches.fr:8080`.

2. Cliquer sur **Save**. Coolify génère le routage Traefik et le certificat Let's Encrypt.
3. Chez le gestionnaire DNS du domaine, créer un enregistrement de type **A** pour ce sous-domaine, pointant vers l'adresse IP du serveur Coolify.

4. Déployer

Cliquer sur **Deploy** (en haut de la ressource), puis suivre les logs du déploiement. Une fois terminé, le site est accessible à l'adresse du sous-domaine.

Dépannage : « dependency failed to start: container database … is unhealthy »

Ouvrir les logs du conteneur `database` (onglet **Logs** de la ressource). Si on y lit :

```
Database is uninitialized and password option is not specified
```

alors `MARIADB_ROOT_PASSWORD` (ou `MARIADB_PASSWORD`) est **vide ou absent** : MariaDB refuse de démarrer sans mot de passe. Pour corriger :

1. Onglet **Environment Variables** : vérifier que `MARIADB_ROOT_PASSWORD`, `MARIADB_PASSWORD` et `APP_SECRET` ont chacune une **valeur** (voir C, étape 2).
2. Cliquer sur **Update** sur chaque ligne modifiée.
3. Cliquer de nouveau sur **Deploy**.

Si le conteneur `database` a déjà démarré une fois avec de mauvais mots de passe, supprimer le volume `database_data` de la ressource avant de redéployer (MariaDB ne relit les mots de passe qu'à la première initialisation). Attention : cela efface les données de la base.

5. MAJ du projet

Après chaque modification du projet Symfony, suivre ces étapes dans l'ordre.

5.1 Tester en local

```
php bin/phpunit
php -S localhost:8000 -t public
```

Vérifier que les tests passent et que le site fonctionne avant de pousser.

5.2 Si les entités Doctrine ont changé : créer la migration

```
php bin/console make:migration
php bin/console doctrine:migrations:migrate
```

- Le fichier de migration généré dans `src/Migrations/` **doit être committé** : il sera exécuté automatiquement au démarrage du conteneur en production (le `CMD` du `Dockerfile` lance `doctrine:migrations:migrate`).
- Ne jamais utiliser `doctrine:schema:update` : les modifications du schéma passent toujours par une migration.
- Les données de la base sont conservées entre les déploiements (volume `database_data`).

5.3 Si des dépendances ont changé

```
composer require vendor/paquet
php bin/console importmap:require nom-du-paquet-js
```

- `composer require` modifie `composer.json` et `composer.lock` : les deux fichiers doivent être committés.
- `importmap:require` modifie `importmap.php` : ce fichier doit être committé. Le dossier `assets/vendor/` n'est **pas** committé : il est régénéré pendant le build par `importmap:install`.

5.4 Pousser les modifications sur GitHub

```
git status
git add .
git commit -m "Description de la modification"
git push origin main
```

Vérifier avec `git status` qu'aucun secret (`.env.local`) n'est ajouté.

5.5 Redéployer dans Coolify

Dans la ressource Coolify :

- dans l'onglet **General**, cliquer sur le bouton **Reload Compose file**
- cliquer sur le bouton **Deploy** (en haut à droite), puis suivre les logs du déploiement.
- Coolify récupère le dernier commit de la branche `main`, reconstruit l'image du service php et redémarre les conteneurs. Les migrations sont appliquées au démarrage.
- Le déploiement n'est pas lancé automatiquement par un `git push` : il faut cliquer sur **Deploy**.
- Si seules des variables d'environnement ont changé (onglet **Environment Variables**, bouton **Update**), aucun `git push` n'est nécessaire : cliquer directement sur **Deploy**.

5.6 Vérifier

- Ouvrir le site à l'adresse du sous-domaine et tester la modification.
- En cas d'erreur, consulter les logs du déploiement, puis les logs du conteneur php dans Coolify.